THESIS REPORT / RESEARCH EDITION

GPU-Accelerated Mathematical Framework for Intelligent Visual Pattern Matching

Shivanshu Gupta | Computer vision, machine learning and GPU computing

Abstract

This research compares two approaches to recognizing handwritten digits: a classical pipeline built from oriented gradients, principal component analysis and an RBF support vector machine, and a convolutional neural network design. It also examines how GPU memory movement and batch size affect the cost of accelerating image operations.

The report documents 353 correct predictions out of 360 held-out images for the classical pipeline. The CNN comparison figure labels its 99.17% value as simulated. This edition presents the classical result and the CNN design separately.

Dataset	Evaluation split	Classical result
1,797 grayscale images, 8 × 8 pixels, digits 0–9	1,437 training images; 360 held-out images	353 / 360 correct (98.06%) as reported

Research questions

1. How can local stroke orientation and a lower-dimensional representation support a classical digit classifier?

2. How does a learned convolutional representation differ from engineered gradients and PCA?

3. Under what conditions does the cost of moving small image tensors outweigh GPU computation savings?

Reading guide

Page 2 covers the two pipelines. Page 3 gives the classical evaluation and its confusion matrix. Page 4 discusses CUDA design, evidence limits and references.

01 / METHODS

Two approaches to the same recognition task

Dataset and preprocessing

The scikit-learn Digits dataset contains grayscale 8 × 8 images across ten labels. The report describes a stratified 80/20 split with seed 42. Pixel intensities from 0 to 16 are divided by 16. In a replication, feature transformations and hyperparameter selection should use the training partition only.

Classical pipeline

Stage	Purpose
Oriented gradients (HOG)	Estimate horizontal and vertical gradients. Accumulate magnitude-weighted votes into nine unsigned orientation bins to describe local strokes.
Principal component analysis	Project the feature representation into a 40-component subspace, as described in the report. Fit the projection using training data.
RBF support vector machine	Classify the projected features using a nonlinear decision boundary. The report lists C = 10 and gamma = scale, with five-fold stratified tuning.

Convolutional network design

The CNN design takes the image as a spatial tensor and learns filters with convolution and ReLU. Pooling reduces spatial dimensions. A dense layer and ten-class softmax produce digit scores. The report describes optimization with categorical cross-entropy and Adam.

Reproduction detail

The reported descriptions of PCA sometimes refer to raw pixel vectors and sometimes to the classical feature array. A replication needs the exact feature construction, CNN padding and pooling configuration, and evaluation split before comparing scores.

Worked operation

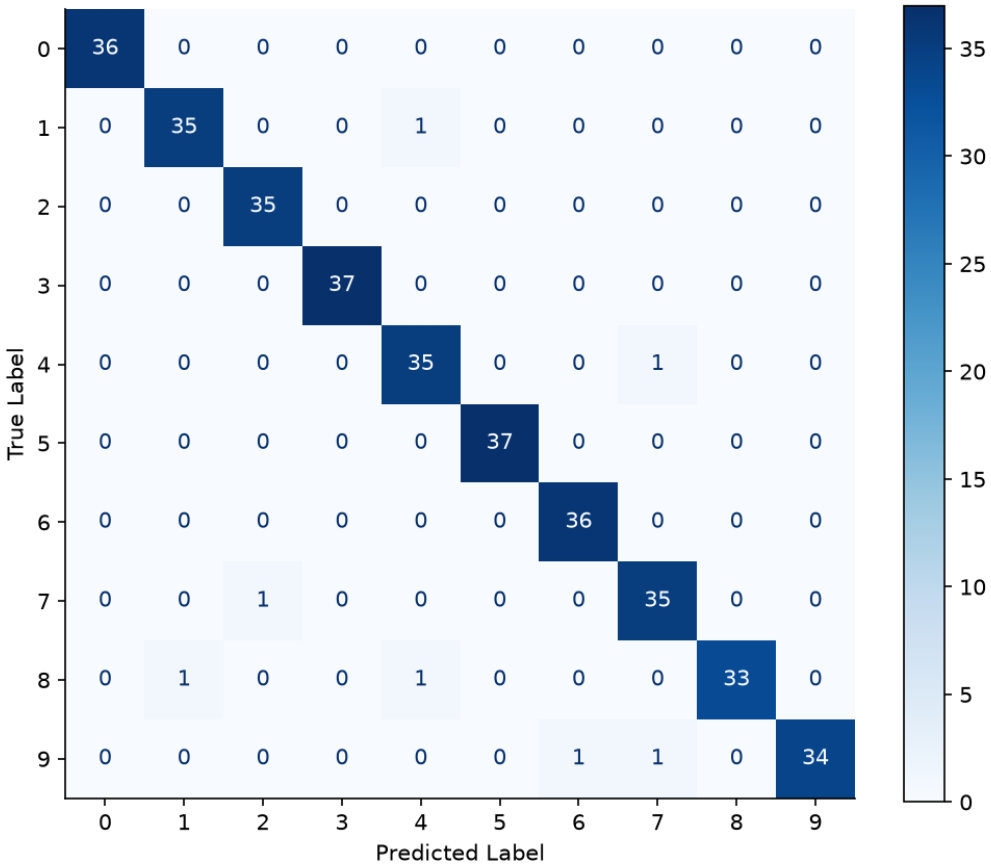
For one 3 × 3 patch, a Sobel-X filter computes a weighted contrast between its left and right columns. This illustrates how directional gradients can turn pixel values into stroke features.

02 / EVALUATION

What the classical result shows

The report's confusion matrix records seven errors on the 360-image held-out partition. The diagonal sums to 353 correct predictions, or **98.06%**. The largest error counts occur for true digits 8 and 9, with two each.

Figure 10 – Confusion Matrix: HOG+PCA+SVM



Confusion matrix from the thesis. Rows are true labels. Columns are predicted labels.

Interpretation

On this split, the classical pipeline recognizes most of the low-resolution digits. The result applies to the reported dataset and split. The report's simulated CNN comparison is insufficient to establish a measured accuracy advantage.

03 / COMPUTE

What changes when work moves to a GPU

Compute and data movement

Convolution and dense layers contain operations that can run in parallel. The thesis studies how CUDA thread blocks and shared-memory tiles can reuse data during matrix multiplication. A practical profile also needs to count kernel launches, host-to-device copies and device-to-host copies.

The batching question

An individual 8 × 8 image contains little computation. Launch and transfer costs can dominate that workload. Batching can amortize fixed costs across more samples, and keeping intermediate tensors resident can reduce copies. The useful batch size depends on hardware, memory pressure and latency targets.

Evidence and next experiment

The thesis uses one low-resolution dataset and a documented classical split. Reproducing a direct model comparison would require the exact training code, configuration and evaluation logs. To test GPU claims, record CPU baseline settings, batch size, timing boundaries and repeated measurements on the same workload.

Selected references

1. N. Dalal and B. Triggs. Histograms of oriented gradients for human detection. CVPR, 2005.

2. C. Cortes and V. Vapnik. Support-vector networks. Machine Learning, 20, 273–297, 1995.

3. Y. LeCun, L. Bottou, Y. Bengio and P. Haffner. Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11), 2278–2324, 1998.

4. F. Pedregosa et al. Scikit-learn: Machine learning in Python. JMLR, 12, 2825–2830, 2011.

5. NVIDIA. CUDA programming documentation.